1 Session I: Foundations and Objects of Mathematical Logic

1.1 Introduction

Logic is a fundamental pillar of computer science, serving as the calculus of computer science. It provides the formal tools needed to express ideas precisely, reason correctly, and verify whether conclusions follow from given information.

In everyday life, humans reason informally using natural language. However, natural language is often ambiguous, imprecise, and context-dependent.

Question: examples for ambiguous, imprecise, and context-dependent. 1. Ambiguity occurs when a single sentence can be interpreted in two or more distinct ways. Example: "I saw the man with the binoculars." The Problem: Did you use the binoculars to see the man? Or was the man you saw holding a pair of binoculars? In formal logic, we use parentheses to eliminate this. A computer needs to know if the property of having binoculars belongs to the subject or the object.

2. Imprecision happens when the terms used don't have a sharp boundary, making it impossible to determine exactly when a statement becomes true or false. Example: "add a few grains of salt to the water." How many is "a few"? Is it 3? 50? 200? In computer science, we define constants. Instead of "a few," we would use a specific integer.

3. Context-dependency, this occurs when the meaning of a statement changes based on who said it, where they are, or when they said it. Example: "it is freezing here today." The Problem is the word "Freezing"; to a Canadian, this might mean -20 C; to someone in Florida, it might mean 10 C. The word "Here" changes based on GPS coordinates. The word "Today" changes every 24 hours. Logic prefers eternal truths, statements that remain true regardless of the speaker's location or time.

Logic replaces this informal reasoning with formal systems governed by strict rules. The objective is to transform intuitive arguments into precise mathematical objects that can be analyzed rigorously. Therefore, logic is the branch of mathematics and philosophy that studies the principles of valid reasoning. It is defined as the study of the methods used to distinguish correct from incorrect reasoning, and it is prescriptive rather than descriptive: logic does not investigate how people actually think, as psychology does, but instead focuses on how one ought to reason in order to reach the truth. Moreover, logic is concerned with the form of reasoning rather than its specific subject matter or content, allowing the same logical principles to be applied across different domains.

Intuitive reasoning is fast, based on experience, feelings, or common sense. However, it is often flawed. Example: "All expensive cars are fast. This car is fast, so it must be expensive." (intuitively tempting but logically invalid).

Logical reasoning is slow and systematic. It is based on strict rules of inference and is guaranteed to be correct if those rules are followed. Example: "If it is a square, it has four sides. This is a square; therefore, it has four sides." (logically valid).

1.2 The Objects of Logic

To build a system of logic, we must define the main objects of study: propositions, truth values, arguments, inference, and validity.

The proposition: A proposition is a declarative statement that is either true or false, but not both. This definition relies on the principle of Bivalence, also known as the Law of Excluded Middle, which states that every proposition must have exactly one truth value, either 1 or 0, with no intermediate possibility. In addition, the Law of Non-Contradiction ensures that a proposition cannot be simultaneously true and false at the same time.

Examples of propositions:

- ✓ "$2 + 2 = 4$" (true)
- ✓ "5 is an even number" (false)
- ✓ "Paris is the capital of France" (true)
- ✓ "7 is prime" (true)

Non-examples (not propositions):

- Commands: "close the door." (no truth value).
- Questions: "is it raining?" (cannot be true or false).
- Open Sentences: "$x + 1 > 3$" (value depends on x; becomes a proposition only when x is defined).

Other examples: Paradox: this statement is false. Exclamative: what a beautiful picture! Performative: I promise to come. Subjective (both true and false): this room is hot. If you are sitting under an AC vent, it is false for you. If I am wearing a wool sweater, it is true for me. The word Hot is a subjective quality and not a measured value. Suggestion: you should revise the lesson. Incomplete: because he is here. Vague: that (?) is large. Also, the Half-Truth in Fuzzy Logic: The glass is full. Quantum Logic: superposition, etc.

Truth values: the truth value of a proposition is its status as true (T/1) or false (F/0).

The argument: An argument is a sequence or collection of propositions consisting of:
Premises: Propositions offered as evidence, assumptions, or reasons.
Conclusion: The proposition claimed to follow logically from the premises.
Structure of an Argument:

Premise 1
Premise 2
_____
Conclusion

Standard example:

Premise 1: All humans are mortal.
Premise 2: Socrates is a human.
_____
Conclusion: Socrates is mortal.

Question: Premise 1, 2 are Assumption/Evidence/Reason? Why?, Premise 1 is A, Premise 2 is E. Premise 1 does not explain why Socrates is mortal. Question: can we make Premise 1 a reason? Yes: if someone is human, he is mortal. Question: example containing A, R, and E! A: the blackboard is made of metal. R: metals expand when the temperature increases. E: temperature of our blackboard increases. C: our blackboard expands. Question: R/A summary! R: when justifies the inference. A: when accepted without justification. Note: the proposition can be A or R depending on the context.

Inference (the process): Inference is the linking step, the movement of the mind from premises to the conclusion using rules of reasoning.

Informal inference based on intuition, language, and common sense. Example: "It is raining, so the ground is wet." Formal inference based on symbols and strict mathematical rules. CS relies exclusively on formal inference so that reasoning can be checked, automated, and implemented.
Example:

$$P \to Q, \quad P \vdash Q$$

Modus Ponens: if $P, P \to Q$ then Q. $P \vdash Q$: syntactic entailment, i.e., there exists a formal proof of A from P (Modus Ponens, generalization...).

1.3 Validity and Truth

This distinction is crucial for rigorous logic.
Truth: A property of individual propositions (factual correctness).
Validity: A property of arguments. An argument is valid if: *whenever all premises are true, the conclusion must also be true.*

Important note: An argument can be valid with false premises or invalid with a true conclusion. Logic studies the validity of the structure, not the factual truth.
Example of valid but false content:

Premise: All birds can fly. (false)
Premise: Penguins are birds. (true)
Conclusion: Penguins can fly. (false)

The reasoning is perfect (valid), even though the content is factually incorrect.

Question: example of an invalid argument with a true conclusion! Some birds can fly; penguins can fly; penguins exist.

1.4 Syntax and Semantics

Syntax (the grammar): The rules for constructing formulas and strings of symbols.
Analogy: In Python, `if x > 5:` is syntactically correct. `if > 5 x` is a syntax error.
Semantics (the truth): The meaning or truth value of those formulas.
Analogy: `Price = 50` is semantically true. `Price = -50` is semantically false.

1.5 The Role of Logic in Computer Science

Formal logic allows us to eliminate ambiguity and to verify correctness mechanically and rigorously, making it foundational to computer science. It underpins digital circuit design, where hardware transistors operate according to Boolean logic with values 0 and 1. Question: Example! to add two numbers, a circuit called a Half-Adder uses an XOR gate to find the sum and an AND gate to determine if there is a carry digit.

And it is essential for program specification and verification, enabling critical software systems such as avionics and medical devices to be proven correct rather than merely tested. Question: Example! testing only proves that a program works for the specific tried inputs (e.g., I checked 1, 2, and 5, and they worked). Formal verification uses mathematical proofs to show it works for all possible inputs; e.g., before a line of code runs, we define a precondition (P). After it runs, we define a post-condition (Q). If we can mathematically prove that $P \to Q$ always holds true, the software is proven correct.

Logic also lies at the heart of algorithm design, since every if-then-else construct is a logical conditional. Question: Example! IF (Balance $\geq$ Withdrawal) THEN (Dispense Cash) ELSE (Show Error). The computer evaluates the statement Balance >= Withdrawal as a Boolean (True/False). If the logic isn't airtight (e.g., forgetting to check if the account is frozen), the algorithm fails.

And it forms the basis of database theory, where SQL queries rely on predicate logic. Question: Example! SQL: SELECT Name FROM Users WHERE Age > 21; Logic equivalent: $\{u.Name \mid u \in Users \wedge u.Age > 21\}$The database engine treats the query as a logical formula and finds all constants (data rows) that make that formula True.

Furthermore, formal logic is central to artificial intelligence through knowledge representation and automated reasoning. Question: Example! before modern black-box AI (like LLMs—Large Language Models), AI was built on inference engines. We represent facts as logical statements and let the computer infer new ones.

And to cybersecurity, where security protocols and cryptographic schemes are validated using formal logical proofs. Question: Example! in a Zero-Knowledge Proof, logic is used to prove that Person A knows the password without Person A actually sending the password over the internet. We use logical symbols to prove that no information is leaked during the exchange.

1.6 Summary

In this session, we transformed human reasoning from informal "gut feelings" into formal mathematical objects. By understanding propositions, arguments, and the difference between truth and validity, we establish the foundation for the entire study of logic in computer science.

Go to Exercises

2 Session II: Syntax and Semantics

2.1 Introduction

To reason rigorously and to represent reasoning without the risk of interpretation errors, we rely on formal languages. A formal language is based on a fixed alphabet, meaning it consists of a finite and well-defined set of symbols. These symbols are combined according to strict formation rules that are mathematically precise, ensuring that only well-formed expressions are constructed. Consequently, every expression in a formal language has an unambiguous meaning, with exactly one valid interpretation. In addition, formal languages are computable, which means that their expressions can be processed by machines mechanically, without any need for cultural understanding, intuition, or contextual interpretation.

In the previous session, we introduced logic as a formal study of reasoning. However, to reason formally, we must transition from the "what" of logic to the "how." This requires answering a fundamental question: How do we precisely describe expressions and their meaning?

This passage shows the transition from philosophy to engineering. It is the moment where we stop talking about thinking vaguely and start building a mathematical system that a machine can actually use. Question: Example of what and how! it is like the difference between knowing "what" a cake is (tasty, sweet, baked) and having a precise recipe (the "how"). Consider the following examples:

1. Music: the What is knowing that a song is happy and played on a piano. We can recognize it, but we cannot replicate it perfectly. The How is sheet music. SSS? The expressive symbols are the notes A-G, syntax is time signatures and bars, and semantics is the specific frequency of sound each note produces. A machine (a player piano) can read the How and play the song perfectly every time.

2. Directions: the What is telling someone, The house is north of the park, near the big tree. It is a vague idea that might get them lost if they do not know which tree we mean. The How is providing GPS coordinates; this is a formal system. SSS? The expressive symbols are numbers, letters, etc. (36°45'10.4"N 3°02'31.2"E), the syntax is the degree/minute/second format (it has many formats); and the semantics is a single and unambiguous point on the planet that a computer can find.

3. Color: the What is describing a color as Sky Blue. Different people will imagine different shades, some lighter, some darker. The How is defining the color as a hex code $\#87CEEB$. SSS? The expressive symbols are the number sign (hash) plus hexadecimal (0-F), the syntax is $\#XXXXXX$, and the semantics are the exact intensities of red, green, and blue light. Every computer screen in the world will show the exact same color.

That leads to a crucial distinction in logic and computer science:

Syntax $\rightarrow$ The form and structure of expressions.
Semantics $\rightarrow$ The meaning and truth value of expressions.

2.2 Syntax: The Anatomy of a Formula

The syntax is the grammar of logic. In everyday language, we are messy; we use slang, tone, and context. In formal logic, we need to know exactly what a legal sentence looks like so a computer can process it without getting confused.

The "What": "If it snows, the weather is cold." The "How" (syntax): we define symbols like P, Q, and $\rightarrow$. We create a rule that says $(P \rightarrow Q)$ is a valid expression, but $(\rightarrow PQ)$ is gibberish.

Therefore, syntax is the part of logic that studies the form and structure of expressions, independently of their meaning. It answers questions such as: is this expression well-formed? Or is it written correctly according to the rules?

The alphabet of propositional logic (PL): every formal language begins with an alphabet Question: What is an alphabet? An alphabet (often denoted by the Greek letter Σ, sigma) is a finite, non-empty set of symbols used to build the expressions of a formal language. The alphabet is the raw material. By themselves, they do not have meaning; they are just distinct marks that we agree to use. Example: In digital circuits (Boolean logic), it is the symbols $\{0, 1\}$. In programming languages, it is the symbols $\{a, b, c, \ldots, 1, 2, 3, \ldots, +, -, *, /, \ldots\}$.

In propositional logic, this includes:

- Propositional variables: Symbols like p, q, r, s or P, Q, R, S (correspond to atomic facts).

- Logical connectives/symbols:
  $\neg$ (Unary): Negation ($\sim$ and overline (vinculum) may also be used).
  $\wedge, \vee, \Rightarrow, \Leftrightarrow$ (Binary): Connect two formulas ($\cdot, \&, +, \rightarrow, \leftrightarrow$ may also be used).

- Auxiliary symbols: Parentheses "(" and ")".

The logical connectives are read as follows:
$\neg P$: read as "not P". It denotes the negation of P.
$P \wedge Q$: read as "P and Q". It denotes the conjunction of P and Q.
$P \vee Q$: read as "P or Q". It denotes the disjunction of P and Q.
$P \rightarrow Q$: can be read as "P implies Q", "if P then Q", "Q if P", or "P only if Q". It denotes the implication from P to Q.
$P \leftrightarrow Q$: read as "P if and only if Q", P is equivalent to Q. It denotes the equivalence between P and Q.

Question: Why $P \rightarrow Q \equiv P$ only if Q? i.e., P true if only Q true! P only if $Q = P$ cannot be true unless Q is true. Q is necessary for P, but P is sufficient for Q. Example: "You can graduate only if you pass the exam." So, passing the exam is required for graduation; if you graduate, then you passed the exam. Thus, P: "You graduate", Q: "You pass the exam." Why is it NOT $Q \rightarrow P$? The sentence does not say that passing the exam guarantees graduation. You might pass the exam, but still not graduate (other requirements).

If we do not have a precise way to describe the expressions and their meaning, we cannot perform automated reasoning. To a computer, reasoning is just a series of mechanical steps.
Input: see an expression (syntax).
Evaluate: determine its truth value based on rules (semantics).
Output: produce a new, proven expression.

Formation rules (the grammar) : Formation rules specify how symbols can be combined to form valid expressions. A string of symbols that follows these rules is called a Well-Formed Formula (WFF):
Rule 1: Any propositional variable is a WFF.
Rule 2: If A is a WFF, then $(\neg A)$ is a WFF.
Rule 3: If A and B are WFFs, then $(A \wedge B), (A \vee B), (A \Rightarrow B)$, and $(A \Leftrightarrow B)$ are WFFs.
Rule 4: Nothing else is a WFF.
So, rules 1-3 tell us how to make sentences correctly, like "The cat sleeps" or "John runs fast". Rule 4 says any string of words that does not follow these patterns is not a valid sentence.

Examples: assume p and q are propositional variables.
✓ $(p \wedge q) \Rightarrow r \rightarrow$ syntactically correct.
✓ $\neg p \rightarrow$ syntactically correct.
✗ $\wedge pq \rightarrow$ syntactically incorrect.
✗ $(p \neg q) \rightarrow$ syntactically incorrect.
✗ $p \wedge \vee q \rightarrow$ syntactically incorrect.
Question: Other examples!

Important note: at this stage, meaning does not matter. Syntax only defines what can be written.

2.3 Semantics: Bringing Formulas to Life

Semantics is the truth value of logic. Once we have a valid expression, we need a rigorous way to decide if it is true or false. This is not based on opinion; it is based on mathematical mappings (relative to a model).

The "What": deciding if the statement about the weather is true. The "How" (semantics): we use truth tables; e.g., we define that $(P \rightarrow Q)$ is only false if P is true and Q is false. Otherwise, it is mathematically true. This removes human feelings from the equation.

Therefore, semantics is the part of logic that studies the meaning of syntactically correct expressions. It does not ask whether a statement is true in the real world; it answers: is this formula true or false?

A formula can be syntactically correct but semantically false. Question: Examples! In propositions: $p \rightarrow q$ is syntactically valid even if p = "The moon is cheese" and q = "2 + 2 = 5". In programming languages/compilers: $x = y/z$ where x is an integer, it is a semantics error. In database systems:

SELECT * FROM Employees WHERE Name > 500; where Name is a string or the column does not exist, etc.

Aspect	Syntax	Semantics
Focus	Form / Structure	Meaning / Truth
Core Question	Is it well-written?	Is it true?
Programming Analogy	Syntax error: missing semicolon; wrong keyword	Semantic error: division by zero; incorrect logic (infinite loop)

2.4 Summary

Understanding syntax and semantics allows us to design formal languages, build compilers, and verify complex systems. This distinction remains central throughout the course.

Go to Exercises

3 Session III: Propositional Logic - Syntax

3.1 Introduction

Propositional Logic (PL) is the simplest and most fundamental logical system. It moves us from the general philosophy of reasoning into a specific, symbolic mechanics. PL studies how propositions, the basic building blocks of thought, can be combined using logical operators to form more complex statements whose truth values can be systematically analyzed. In this session, we transition from informal human expressions to a rigorous symbolic language.

3.2 Propositional Building Blocks

In PL, we treat propositions as a unit of information. We categorize them based on their complexity.

a. Atomic Propositions: An atomic proposition is a simple declarative statement that has a complete thought and contains no explicit logical connectives. Syntactically, it cannot be broken down further without losing the meaning. Semantically, it can be evaluated to a definite truth value. In formal logic, these are represented by propositional variables (typically uppercase or lowercase letters).
Examples:

(p): "The server is inactive."
(q): "The database is offline."
(r): "I know that I am capable."
(s): "I won the contest last week."
(t): "You can have pie."
(u): "The store is open."
(y): "It is a workday."

Question: other examples!

b. Compound Propositions: A compound proposition is formed by combining one or more atomic propositions using logical connectives ($\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$). Its truth value depends entirely on the truth values of its component atoms.
Examples:

"The server is inactive, or the database is offline" $(p \vee q)$.
"I know that I am capable, and I won the contest last week." $(r \wedge s)$
"You cannot have pie." $(\neg t)$
"If the store is open, then it is a workday." $(u \Rightarrow y)$

Question: other examples!
Question: Is this an atomic proposition? The file had not been removed.

Yes, it can be an atomic proposition, provided we interpret it correctly. Although the sentence contains the word "not", this is linguistic negation, not a logical connective in formal language.

Let p: "The file had not been removed." We do not rewrite it as $\neg$(file was removed) unless we explicitly choose to model it that way. Atomic propositions may contain words like "not", "and", or "or" in natural language, but in propositional logic, they are treated as indivisible symbols unless explicitly formalized.

Question: How to differentiate between linguistic negation and the logical connective "not"? (respectively, the other connectives)

Linguistic negation: appears inside an atomic proposition and has no formal meaning in logic unless we explicitly translate it (appears before formalization). Logical negation: appears outside atomic propositions and always applies to a well-formed formula (appears after formalization).

Suppose: p: "the file is removed.", the implicit negation is: q: "the file is not removed." and the explicit negation is: r: "It is not the case that the file is removed." Therefore, r is a compound proposition, and q is an atomic proposition unless we have p.

Question: Examples of linguistic and/or inside atomic propositions

Research and development are costly. Coffee or tea is available. 'Bread and butter' is a tradition. The toy's color is red and blue.

The same thing here, if we have p: "research is costly." q: "development is costly." then the explicit And is: "research is costly, and development is costly." and the implicit And is r: "Research and development are costly". So r is atomic unless we have p and q.

3.3 The Translation Map (Natural Language to Logic)

3.4 Logical Connectives in Translation

Connectives are the operators of logic. In computer science, they function similarly to arithmetic operators $(+, -, \times)$, but they operate on truth values (Boolean values).

Negation

Symbol:$\neg$, $\sim$, or an overline
Role: A unary operator (affects only one formula). It reverses the truth value.
Natural Language: "not", "it is false that", "it is not the case that".
Question: examples! let q: "The door is open." Negation: "The door is not open." "It is false that the door is open." "It is not the case that the door is open."

Natural-language expressions that may correspond to the negation ($\neg$): direct negation: no, not, never, none, nothing, nowhere; lexical (word-level): false, incorrect, wrong, untrue; negative verbs: deny, refuse, fail to; prefix-based: un- (unhappy, unfair), in-/im-/il-/ir- (invalid, impossible, illegal, irrelevant), non- (nonempty, nonzero); modal-style: cannot, must not, is impossible that; contrastive negation (implicit): instead of, rather than, etc. If a sentence asserts that a proposition does not hold, regardless of wording, it is modeled as negation in propositional logic.

Conjunction

Symbol: $\wedge$, $\cdot$, or &
Role: A binary operator. It is true only if both components are true.
Natural Language: "and", "but", "moreover", "while", "although".
Note: In logic, "but" is treated the same as "and" because both imply that both facts are simultaneously true.
Question: examples! Let p: "Alice passed the exam." and q: "Alice submitted the project." Conjunction: "Alice passed the exam and submitted the project.", "Alice passed the exam; moreover, she submitted the project.", "Alice passed the exam but submitted the project."

Natural-language expressions that may correspond to the conjunction $\wedge$: as well as, also, together with, moreover, furthermore, in addition, additionally, but, yet, however (not to oppose arguments), although, though, even though, despite the fact that, etc.

Disjunction

Symbol: $\vee$ or $+$
Role: A binary operator. It is true if at least one component is true.
Natural Language: "or", "unless".
Critical Distinction: This is the Inclusive OR (A or B or both). In computer science, this differs from the Exclusive OR (XOR), which is false if both are true.
Question: examples! "I will go to the park unless it rains". The statement says: I go to the park if it does not rain, it can be rewritten as "I will go to the park or it rains", showing a disjunction: you go to the park, or it rains (or both). The statement is false only if I do not go to the park and it does not rain. In all other cases (I go, it rains, or both), it is true. The logic only guarantees I will be there if the rain does not happen. (if $\neg$ 'rain', then 'go'; if $\neg$ Q then P; $\neg$ Q $\Rightarrow$ P; Q $\vee$ P).

Natural-language expressions that may correspond to the disjunction $\vee$: "at least one of A and B", "A alternatively B", "A in case B" (Take an umbrella in case it rains), etc.

Implication

Symbol: $\Rightarrow$ or $\rightarrow$
Role: Represents a conditional relationship.
Structure: In $p \Rightarrow q$, p is the antecedent (hypothesis) and q is the consequent (conclusion).
CS Context: This is the logic behind `if (condition) { action }`.
Question: examples! Let p = "I study hard" and q = "I will pass the exam". The implication $p \Rightarrow q$ reads as: "If I study hard, then I will pass the exam."

Natural-language expressions that may correspond to the implication $\Rightarrow$: "q if p", "p implies q", "Whenever p, q", "q whenever p", "On the condition that p, q", "q provided that p", etc.

Biconditional / Equivalence

Symbol: $\Leftrightarrow$ or $\leftrightarrow$
Role: True only if both components have the same truth value.
Natural Language: "if and only if" (iff), is equivalent to "just in case".
Question: examples!

Let $p =$ "You are a bachelor" and $q =$ "You are an unmarried man".
The biconditional $p \Leftrightarrow q$ reads as: "You are a single if and only if you are an unmarried man." This means: If you are a single, then you are an unmarried man ($p \Rightarrow q$). If you are an unmarried man, then you are a single ($q \Rightarrow p$).

Natural-language expressions that may correspond to the equivalence $\Leftrightarrow$: "p is equivalent to q", "p just in case q", "p exactly when q", "p holds precisely when q holds", "p is true exactly if q is true", etc.

You can watch TV if and only if you finish your homework. Two triangles are congruent just in case all their corresponding sides and angles are equal.

Important Note: Most linguistic connectors are semantically flexible and therefore do not map uniquely to a single logical connective and may be translated into different logical connectives depending on their semantic role and context ("You touch that dial, and you will get an electric shock" it is $P \to Q$). For example, the translation may depend on whether you are describing two things happening or stating a rule: e.g., "The bird sang while the sun shone." $P \wedge Q$, "Cars stay stopped while the light is red. $Q \to P$.

Other linguistic connectors: while, when, at the same time that, then, because, which explains, so, since, therefore, as long as, etc.

Question: examples!

3.5 Formalization

One of the most challenging tasks is mapping human speech to logical symbols. Formalization is the process of translating a problem from natural language into a formal language. This step is crucial because it introduces precision by eliminating noise and ambiguity, leaving only the underlying logical structure of the problem.

Connective	Natural Language Cues	Logical Form
Negation	"It is not true that...", "None", "Never"	$\neg p$
Conjunction	"p although q", "p but q", "p while q"	$p \wedge q$
Disjunction	"p or q (or both)", "p unless q"	$p \vee q$
Implication	"q if p", "p is sufficient for q", "p only if q"	$p \Rightarrow q$
Equivalence	"p is necessary and sufficient for q"	$p \Leftrightarrow q$

Enriched explanation: Neither P nor Q is written as $\neg(P \vee Q)$, a negated disjunction. The "If" and "Only If" trap:
"A if B": This means B is the condition for A. Translation: $B \Rightarrow A$.
"A only if B": This means A cannot be true without B being true. Translation: $A \Rightarrow B$.
Sufficient vs. Necessary: If p is sufficient for q, it means $p \Rightarrow q$. If p is necessary for q, it means $q \Rightarrow p$.

3.6 Summary

In Session III, we defined the core components of propositional logic. We introduced atomic and compound propositions. We also covered the five main logical connectives. Finally, we developed translation skills to convert complex natural-language statements into precise symbolic formulas.

Go to Exercises

4 Session IV: Variables, Well-Formed Formulas, and Substitution

4.1 Introduction

This section delves more into the foundational building blocks of propositional logic: propositional variables and logical constants as symbolic representations of truth, well-formed formulas as grammatically valid expressions, parentheses and precedence as tools for unambiguous structure, and substitution as a method for transforming formulas. Together, these concepts define what counts as a legitimate logical expression and how such expressions can be manipulated without altering their logical meaning.

4.2 Propositional Building Blocks: Variables and Constants

Question: Does a propositional variable represent a sentence or the truth value of that sentence?

In algebra, we use x and y to represent unknown numbers. In logic, we use propositional variables to represent unknown truth values. We touched upon the concept of a variable and said that it is a symbol corresponding to an atomic fact in an informal explanation.

It is a symbol (usually p, q, r, s or $p_1, p_2, \dots$) that stands for an arbitrary atomic proposition. The domain of these variables is strictly the set $\{True, False\}$ (or $\{1, 0\}$). Variables allow us to manipulate propositions symbolically and discuss the form of an argument regardless of its specific content.

Logical Constants (Verum and Falsum): Formal logic often includes two special constant symbols that represent fixed values:
$\top$ (Top / Verum): A constant that is always true.
$\bot$ (Bottom / Falsum): A constant that is always false.
$v(\top) = 1$ (true formula),
$v(\bot) = 0$ (false formula).
The truth constants Top ($\top$) and Bottom ($\bot$) are also called logical constants.

Question: example of an absolute truth and an absolute falsehood?

"$1 = 1$" or "The sun exists". "$1 = 2$" or "A circle has four corners".

Question: If $\top$ is always true, why do we even need it? Why not just write "true"?

Ans: to stay inside the formal language and to treat truth like a symbol, this shows that logic is a symbolic system.

Question: What are the following equivalent to? $p \wedge \top$, $p \wedge \bot$, $\bot \implies p$, $p \implies \bot$

They are: p, $\bot$, $\top$, and $\neg p$.

4.3 Well-Formed Formulas (WFF)

As stated earlier (see: Formation rules), a well-formed formula is a string of symbols that is grammatically correct according to the syntax of propositional logic. This is analogous to a syntax check performed by a compiler; if an expression is not a WFF, it has no logical meaning.

The definition of WFFs is inductive. Consequently, properties of formulas are typically proved by structural induction, where the base case handles atomic formulas and the inductive steps handle negation and binary connectives.

So, the grammar-based definition (equivalent view) is:

$$\phi ::= p \mid \top \mid \bot \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \Rightarrow \phi) \mid (\phi \Leftrightarrow \phi)$$

where p ranges over propositional variables.

Question: Why do you think WFFs are defined recursively instead of all at once?

Ans: to build complex formulas from simple ones.

Question: other examples of a recursive definition?

Fibonacci Sequence: in a recursive definition, we define the base cases (the starting points) and then define the recursive step (how to build the next piece using the previous pieces). The Fibonacci sequence F_n is defined recursively as follows: Base cases: $F_0 = 0, F_1 = 1$; Recursive step: $F_n = F_{n-1} + F_{n-2}$. Factorials: $n! = n \times (n-1)!$ with the base case $0! = 1$.

4.4 Parentheses and Precedence

Parentheses in propositional logic explicitly determine the scope of logical connectives and guarantee unambiguous parsing of formulas. In the strict syntactic definition of WFFs, parentheses are mandatory for all compound formulas. However, in practice, excessive parentheses reduce readability. To address this, a system of precedence rules is adopted as a notational convention.

Without precedence rules, every compound formula would require full parenthesization. For example, $((p \wedge q) \Rightarrow (\neg r))$ is syntactically precise but visually cumbersome. Precedence rules allow this formula to be written more compactly as $p \wedge q \Rightarrow \neg r$ while preserving a unique expression.

Operator precedence: A precedence relation is a total order on logical connectives that determines how formulas are grouped in the absence of parentheses. A connective with higher precedence binds its operands more tightly than a connective with lower precedence.

Precedence rules (highest to lowest): $\neg$, $\wedge$, $\vee$, $\Rightarrow$, then $\Leftrightarrow$.
Question: Why should negation bind tighter?
Ans: Because it affects only one formula. $\wedge$ is more restrictive and expresses stronger conditions than $\vee$. $\Rightarrow$ is not a basic logical operation; it is defined using $\vee$. $\Leftrightarrow$ is defined using implications.

Note: $\neg p \wedge q$ is interpreted as $(\neg p) \wedge q$, whereas $\neg(p \wedge q)$ negates the entire conjunction.
Parentheses override precedence: whenever parentheses are present, they take absolute priority over precedence rules. $\neg(p \wedge q) \Rightarrow r \neq \neg p \wedge q \Rightarrow r$
Question: example!
Let: p: You neglect the lesson, q: You save money, r: You are allowed to travel.
$\neg(p \wedge q) \Rightarrow r$: If it is not the case that you both "neglect the lesson and save money," then you are allowed to travel; i.e., if you did not neglect the lesson, or if you did not save money, then you are allowed to travel.
$\neg p \wedge q \Rightarrow r$: If you did not neglect the lesson, but you did save money, then you are allowed to travel. This rule is much more specific: you must not neglect the lesson, and you must still save money.
Associativity conventions:

- $\wedge$ and $\vee$ are associative: $p \wedge q \wedge r \equiv (p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ Question: What about $\Rightarrow$?
- $\Rightarrow$ is right-associative: $p \Rightarrow q \Rightarrow r \equiv p \Rightarrow (q \Rightarrow r)$
- $\Leftrightarrow$ is usually treated as associative or fully parenthesized when ambiguity matters.

Precedence rules correspond to the construction of a syntax tree in which higher-precedence connectives appear deeper in the tree. This guarantees a unique hierarchical structure for any well-written formula. Precedence rules are conventions, not logical laws. Misunderstanding them can lead to incorrect expression, especially with implications.
e.g., let $p \Rightarrow q \wedge r$, correct expression: $p \Rightarrow (q \wedge r)$, incorrect expression: $(p \Rightarrow q) \wedge r$
Question: Example for that!
Let: p: You win the lottery, q: You quit your job, and r: You buy a boat.
1. The correct expression: $p \Rightarrow (q \wedge r)$
"If you win the lottery, then you will both quit your job and buy a boat." In this version, winning the lottery is the trigger for both actions. If you do not win the lottery, you do not necessarily do either. Your boss is safe, and you are still boatless, but the logic remains intact; q and r both are dependent on p.
2. The incorrect expression: $(p \Rightarrow q) \wedge r$
"If you win the lottery, you will quit your job, AND you are buying a boat (no matter what)."
This is a very different life choice! Here, the requirement to buy a boat (r) is no longer attached to winning the lottery. You are buying that boat regardless of your bank account balance. The only thing dependent on the lottery is whether or not you quit your job; q is dependent on p; r is a standalone fact that is true regardless of p.

4.5 Substitution in a Formula

Substitution is the process of replacing a propositional variable in a formula with another formula.
Let F be a formula containing the variable p. The notation $F[p \leftarrow G]$ (or $F[G/p]$) means: Replace every occurrence of p in F with the formula G.

The replacement must be simultaneous. We cannot change one instance of p while leaving another unchanged.

Example: Given the formula $F = (p \Rightarrow q) \wedge p$. Suppose we substitute p with $(r \vee s)$:
Locate all p's: $(\underline{p} \Rightarrow q) \wedge \underline{p}$
Replace them: $((r \vee s) \Rightarrow q) \wedge (r \vee s)$

Why is this essential? If we prove that a formula like $(p \vee \neg p)$ is always true, the Substitution Theorem tells us that replacing p with any complex formula, no matter how large, will also be true. This is a core technique in logic programming and automated theorem proving (Password is correct $\Rightarrow$ Access is granted. Substitute "Password is correct" with "typed from X and starts by Y").

The recipe template: we have a master recipe formula (F) for a basic meal. The variable p stands for protein. The original formula F is: $F = (\text{Season } p) \wedge (\text{Grill } p \text{ for 10 minutes})$. In this formula, p is a placeholder; we need a specific ingredient. Let $G = (\text{Chicken})$, so $F[p \leftarrow G]$ gives (Season *Chicken*) $\wedge$ (Grill *Chicken* for 10 minutes).

Let p: The traffic light is green, q: Cars are allowed to go, and r: The road is clear. Let $F = p \Rightarrow q$ (If the traffic light is green, then cars are allowed to go). Let $G = p \wedge r$ (The traffic light is green and the road is clear). Substituting p in F with G gives: $(p \wedge r) \Rightarrow q$, which reads: If the traffic light is green and the road is clear, then cars are allowed to go.

Question: Which of today's concepts would completely break automated reasoning if it were missing?

Ans: WFFs, substitution, and precedence rules.

4.6 Summary

We defined well-formed formulas inductively, ensuring that every formula has a unique syntactic structure. This recursive definition is essential for formal proofs, parsing, and automated reasoning. We also introduced parentheses and precedence rules as notational conventions that improve readability while preserving unambiguous expression. Parentheses always override precedence, and misunderstanding these conventions, especially for implication, can lead to incorrect expression.

Finally, we studied substitution as the process of replacing propositional variables with formulas. Substitution allows general logical principles to be instantiated with complex expressions and forms a core mechanism in proof systems, logic programming, and automated theorem proving.

Go to Exercises